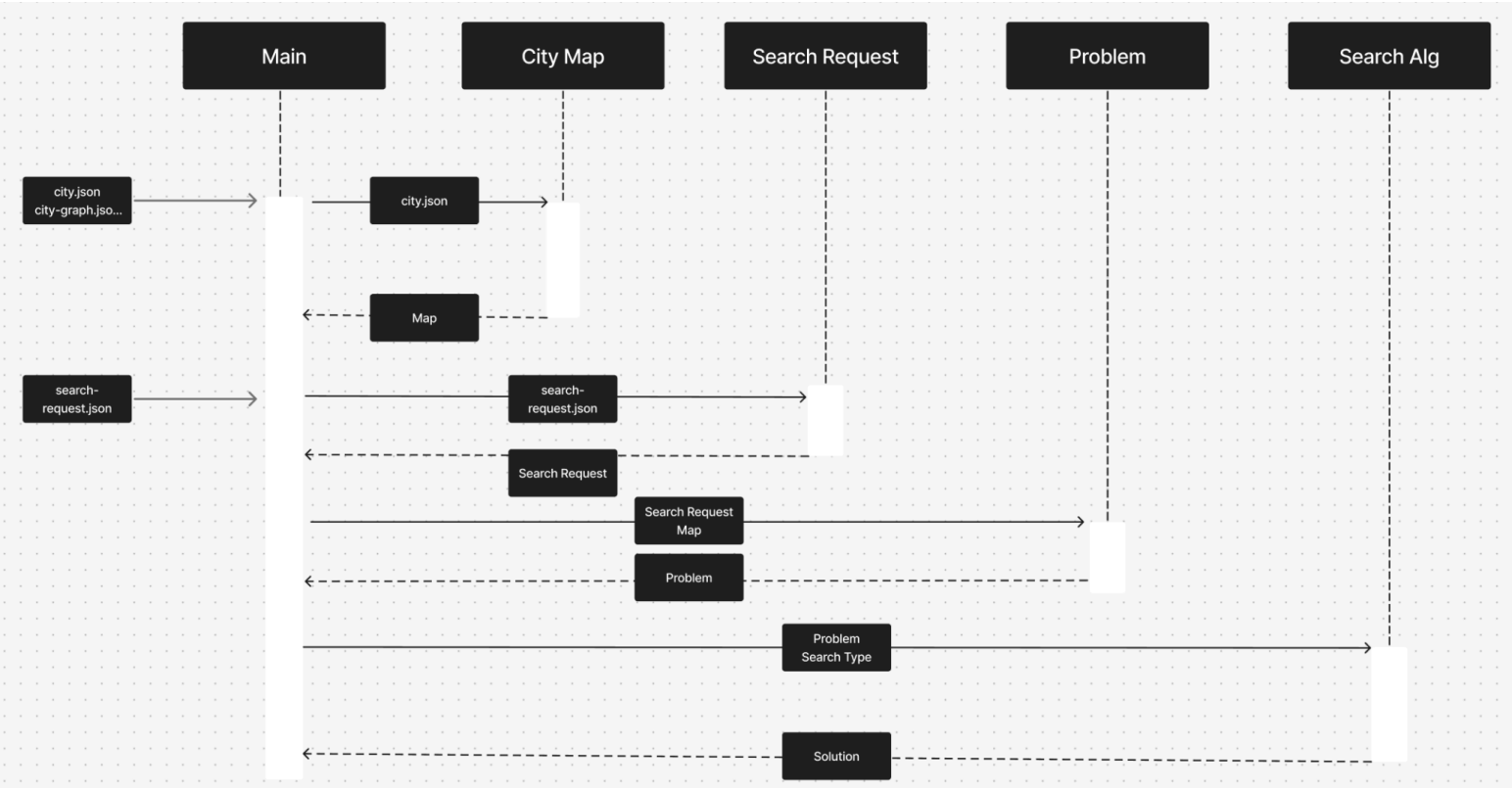
Introduction

The goal of this assignment was to use three different search algorithms to find an efficient route for executing a series of deliveries to locations on a given map—in this case, the city of Tegucigalpa. The list of target delivery locations is referred to as a search request. The assignment required the implementation of three search algorithms: breadth-first search (BFS), uniform-cost search (UCS), and A*. It also included implementing the heuristic function used in A*. Additionally, we implemented several pre-defined classes that enabled us to use the search algorithms to execute a given search request.

Code architecture

The program requires two inputs: a city map and a series of search requests. A single search request contains all locations that must be visited on the route. With the city map, we define the map on which to execute the search request. The table below summarizes this information.

Name	Description	Feilds
search request	represents a search to be executed	• Starting location • Delivery location targets
city map	Simplified map of the city	• Names of each location on the map • Cost of moving from one location to the next



Above is a sequence diagram of the program given a city map and one search request. In this diagram, there is a search algorithm class, which can execute any of the search algorithms: BFS, UCS, or A*. (sequence diagram link)

Below is a short description of each search algorithm

Name	Desciptions
Breadth First Search	Traverse the graph using a FIFO queue to explore nodes and add their neighbors to the queue
Uniform Cost Search	Traverse the graph using a priority queue, which is ordered by the cost of moving from one location to its neighbor
A* Search	Traverse the graph using a priority queue, which is ordered by the cost of moving from one location to its neighbor and the heuristic function.

The heuristic used in A* is calculated using the straight-line distance between locations, given two scenarios

- When there are no deliveries left:
  - Straight line distance from the current location to the start location.
- When there is 1 or more deliveries left:
  - Straight line distance from the current location to either the nearest or farthest delivery location + straight line distance between the closest and farthest delivery locations + straight line distance from the closest delivery location back to the start.

Computer Specs.

The following results were executed on a 2021 MacBook Pro.

OS	Chip	CPU
MAC OS 26.3.1	Apple M1 Max	64 GB

Results.

Test results

Name	Algorithm	Solution	Cost	Nodes	Nodes	Search Time

				Explored	Reached	
Easy 1	BFS	[A1] -> C3 -> (C2) -> C3 -> [A1]	5.454750493	28	40	0.0002863407135
Easy 1	UCS	[A1] -> C3 -> (C2) -> C3 -> [A1]	5.454750493	43	56	0.0004987716675
Easy 1	A*	[A1] -> C3 -> (C2) -> C3 -> [A1]	5.454750493	5	14	0.00008893013
Easy 2	BFS	[M1] -> D3 -> (D1) -> D3 -> [M1] -> M2 -> (R3) -> M2 -> [M1]	7.150659538	209	253	0.001890659332
Easy 2	UCS	[M1] -> M2 -> (R3) -> M2 -> [M1] -> D3 -> (D1) -> D3 -> [M1]	7.150659538	183	229	0.002176046371
Easy 2	A*	[M1] -> M2 -> (R3) -> M2 -> [M1] -> D3 -> (D1) -> D3 -> [M1]	7.150659538	16	41	0.0002710819244
Easy 3	BFS	[D1] -> A1 -> (A2) -> A1 -> C3 -> (C2) -> C3 -> [D1] -> (D3) -> [D1]	10.94744969	329	420	0.003064870834
Easy 3	UCS	[D1] -> (D3) -> D4 -> C4 -> (C2) -> C3 -> A1 -> (A2) -> A1 -> [D1]	10.14243441	400	490	0.00513792038
Easy 3	A*	[D1] -> (D3) -> D4 -> C4 -> (C2) -> C3 -> A1 -> (A2) -> A1 -> [D1]	10.14243441	40	83	0.0008060932159
Medium 1	BFS	[Q1] -> M4 -> M1 -> D3 -> (D1) -> D2 -> I3 -> I4 -> R1 -> (R2) -> L4 -> V2 -> (V4) -> V1 -> Q3 -> Q2 -> (F3) -> Q2 -> [Q1]	19.27682513	1410	1469	0.01377296448
Medium 1	UCS	[Q1] -> M4 -> M1 -> D3 -> (D1) -> D3 -> M1 -> M2 -> R3 -> (R2) -> L4 -> V2 -> (V4) -> V1 -> Q3 -> Q2 -> (F3) -> Q2 -> [Q1]	18.77705747	1405	1467	0.01858115196
Medium 1	A*	[Q1] -> Q2 -> (F3) -> Q2 -> Q3 -> V1 -> (V4) -> V2 -> L4 -> (R2) -> R3 -> M2 -> M1 -> D3 -> (D1) -> D3 -> M1 -> M4 -> [Q1]	18.77705747	402	580	0.009433746338
Medium 2	BFS	[R2] -> L4 -> V2 -> V1 -> Q3 -> Q2 -> F3 -> (F1) -> F2 -> K2 -> (K3) -> O3 -> O2 -> (N3) -> N1 -> M3 -> D4 -> C4	28.52252056	2969	2985	0.02980303764

		-> (C2) -> C3 -> A1 -> (A2) -> A3 -> I3 -> I4 -> R1 -> [R2]				
Medium 2	UCS	[R2] -> R1 -> I4 -> I3 -> A3 -> (A2) -> A1 -> C3 -> (C2) -> C4 -> D4 -> M3 -> N1 -> (N3) -> O2 -> O3 -> (K3) -> K2 -> F2 -> (F1) -> F3 -> Q2 -> Q1 -> M4 -> M2 -> R3 -> [R2]	27.8138299	2938	2959	0.03960299492
Medium 2	A*	[R2] -> R1 -> I4 -> I3 -> A3 -> (A2) -> A1 -> C3 -> (C2) -> C4 -> D4 -> M3 -> N1 -> (N3) -> O2 -> O3 -> (K3) -> K2 -> F2 -> (F1) -> F3 -> Q2 -> Q1 -> M4 -> M2 -> R3 -> [R2]	27.8138299	443	638	0.01197385788
Medium 3	BFS	[L3] -> I4 -> (I1) -> I3 -> D2 -> D3 -> M1 -> M4 -> Q1 -> Q4 -> F4 -> U4 -> (U1) -> U2 -> E2 -> (E4) -> W3 -> (X4) -> X1 -> (T4) -> T1 -> S1 -> (S2) -> S1 -> T1 -> T2 -> L2 -> [L3]	32.81642795	5535	5679	0.05714988708
Medium 3	UCS	[L3] -> I4 -> (I1) -> I4 -> [L3] -> L4 -> V2 -> V1 -> Q3 -> W2 -> W4 -> U4 -> (U1) -> U2 -> E2 -> (E4) -> W3 -> (X4) -> X1 -> (T4) -> T1 -> S1 -> (S2) -> S1 -> T1 -> T2 -> L2 -> [L3]	32.50481306	5758	5854	0.08255505562
Medium 3	A*	[L3] -> L4 -> V2 -> V1 -> Q3 -> W2 -> W4 -> U4 -> (U1) -> U2 -> E2 -> (E4) -> W3 -> (X4) -> X1 -> (T4) -> T1 -> S1 -> (S2) -> S1 -> T1 -> T2 -> L2 -> [L3] -> I4 -> (I1) -> I4 -> [L3]	32.50481306	1707	2208	0.04718708992
Hard 1	BFS	[Q2] -> F3 -> F1 -> (J3) -> J2 -> (H2) -> J2 -> J1 -> K2 -> P4 -> P1 -> M3 -> D4 -> D1 -> A1 -> (A2) -> G1 -> (G2) -> G4 -> I1 -> I4 -> L3 -> L2 -> T2 -> T1 -> S1 -> (S3) -> S1 -> T1 -> T4 -> X1 -> X4 -> W3 -> E4 -> (E2) -> E4 -> W3 -> (W2) -> Q3 -> [Q2]	48.00533845	11669	11778	0.1251249313

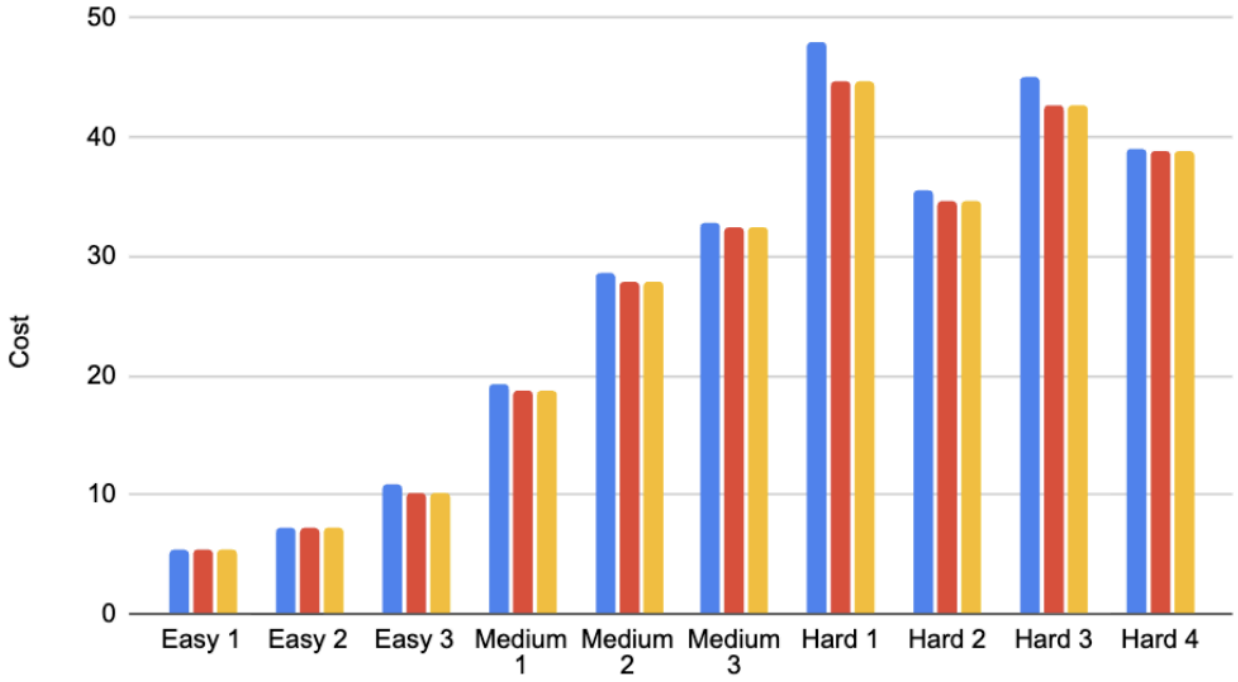
Hard 1	UCS	[Q2] -> Q1 -> M4 -> M1 -> D3 -> D1 -> A1 -> (A2) -> G1 -> (G2) -> G4 -> I1 -> I4 -> L3 -> L2 -> T2 -> T1 -> S1 -> (S3) -> S1 -> T1 -> T4 -> X1 -> X3 -> W1 -> (W2) -> W4 -> U4 -> U2 -> (E2) -> U2 -> U1 -> B4 -> B3 -> (J3) -> J2 -> (H2) -> J2 -> (J3) -> F1 -> F3 -> [Q2]	44.62319432	11635	11721	0.1708140373
Hard 1	A*	[Q2] -> Q1 -> M4 -> M1 -> D3 -> D1 -> A1 -> (A2) -> G1 -> (G2) -> G4 -> I1 -> I4 -> L3 -> L2 -> T2 -> T1 -> S1 -> (S3) -> S1 -> T1 -> T4 -> X1 -> X3 -> W1 -> (W2) -> W4 -> U4 -> U2 -> (E2) -> U2 -> U1 -> B4 -> B3 -> (J3) -> J2 -> (H2) -> J2 -> (J3) -> F1 -> F3 -> [Q2]	44.62319432	6082	6864	0.171849966
Hard 2	BFS	[P2] -> P1 -> N4 -> (N3) -> N1 -> M3 -> (D4) -> (C4) -> C3 -> A1 -> (A2) -> A3 -> I3 -> I4 -> L3 -> L4 -> V2 -> V4 -> (X2) -> X3 -> W1 -> W4 -> F4 -> (F2) -> F1 -> B3 -> (B2) -> B3 -> J3 -> (J2) -> J1 -> K2 -> P4 -> [P2]	35.58337191	23176	23361	0.2549200058
Hard 2	UCS	[P2] -> P4 -> K2 -> (F2) -> F1 -> J3 -> (J2) -> J3 -> B3 -> (B2) -> B4 -> U1 -> U4 -> W4 -> W1 -> X3 -> (X2) -> V4 -> V2 -> L4 -> L3 -> I4 -> I3 -> A3 -> (A2) -> A1 -> C3 -> (C4) -> (D4) -> M3 -> N1 -> (N3) -> N4 -> P1 -> [P2]	34.57345179	23081	23318	0.3685030937
Hard 2	A*	[P2] -> P4 -> K2 -> (F2) -> F1 -> J3 -> (J2) -> J3 -> B3 -> (B2) -> B4 -> U1 -> U4 -> W4 -> W1 -> X3 -> (X2) -> V4 -> V2 -> L4 -> L3 -> I4 -> I3 -> A3 -> (A2) -> A1 -> C3 -> (C4) -> (D4) -> M3 -> N1 -> (N3) -> N4 -> P1 -> [P2]	34.57345179	8008	9741	0.2609539032
Hard 3	BFS	[O1] -> N4 -> N1 -> M3 -> D4 -> C4 -> (C2) -> C3 -> A1 -> (A2) -> G1 -> (G2) -> G4 -> I1 -> (I2) -> I3 -> (D2) -> D4 -> M3 -> P1 -> P4 -> F3 -> (F2)	45.03543065	46679	46789	0.5153417587

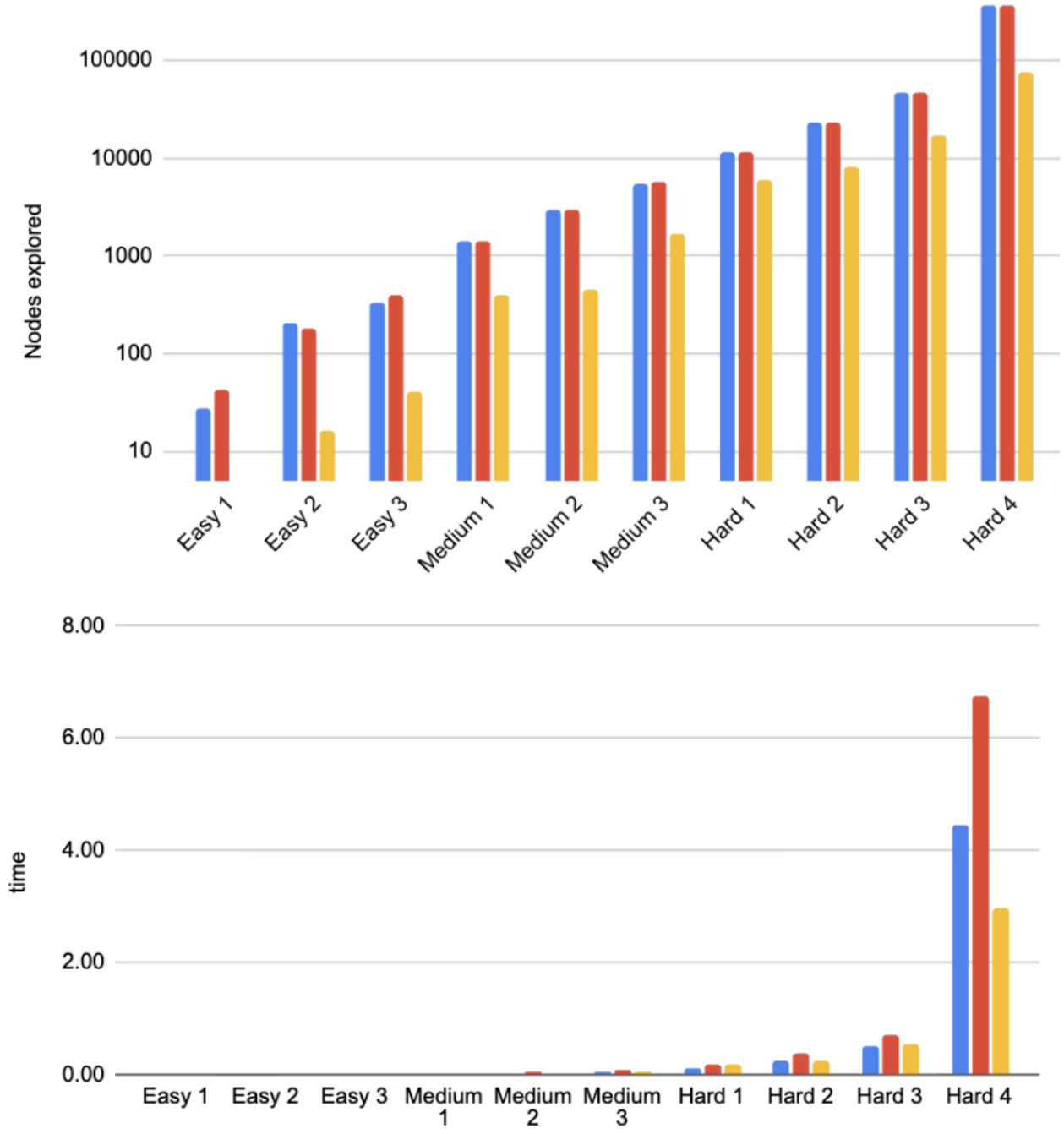
		-> F4 -> U4 -> U2 -> (E2) -> U2 -> U1 -> B4 -> (B2) -> B3 -> J3 -> J2 -> (H2) -> H1 -> J4 -> K4 -> K3 -> O3 -> [O1]				
Hard 3	UCS	[O1] -> P3 -> P4 -> K2 -> (F2) -> F1 -> J3 -> J2 -> (H2) -> J2 -> J3 -> B3 -> (B2) -> B4 -> U1 -> U2 -> (E2) -> U2 -> U4 -> F4 -> Q4 -> Q1 -> M4 -> M1 -> D3 -> (D2) -> I3 -> (I2) -> I1 -> G4 -> (G2) -> G1 -> (A2) -> A1 -> C3 -> (C2) -> C4 -> D4 -> M3 -> N1 -> N4 -> [O1]	42.74651127	46697	46817	0.7050321102
Hard 3	A*	[O1] -> N4 -> N1 -> M3 -> D4 -> C4 -> (C2) -> C3 -> A1 -> (A2) -> G1 -> (G2) -> G4 -> I1 -> (I2) -> I3 -> (D2) -> D3 -> M1 -> M4 -> Q1 -> Q4 -> F4 -> U4 -> U2 -> (E2) -> U2 -> U1 -> B4 -> (B2) -> B3 -> J3 -> J2 -> (H2) -> J2 -> J3 -> F1 -> (F2) -> K2 -> P4 -> P3 -> [O1]	42.74651127	17262	21208	0.5528578758
Hard 4	BFS	[V3] -> V2 -> R4 -> (R1) -> I4 -> (I3) -> A3 -> A2 -> (G1) -> A2 -> A1 -> (D1) -> D4 -> M3 -> P1 -> (P3) -> K3 -> (K1) -> K4 -> (J4) -> H1 -> (H2) -> J2 -> J3 -> B3 -> (B4) -> B3 -> F1 -> (F4) -> W4 -> (W1) -> X3 -> (X1) -> X2 -> V4 -> [V3]	39.04722715	364290	365317	4.446295977
Hard 4	UCS	[V3] -> V4 -> X2 -> (X1) -> X3 -> (W1) -> W4 -> (F4) -> U4 -> U1 -> (B4) -> B3 -> J3 -> J2 -> (H2) -> H1 -> (J4) -> K4 -> (K1) -> K3 -> (P3) -> P1 -> M3 -> D4 -> (D1) -> A1 -> A2 -> (G1) -> A2 -> A3 -> (I3) -> I4 -> (R1) -> R4 -> V2 -> [V3]	38.85845591	356713	361878	6.754083157
Hard 4	A*	[V3] -> V4 -> X2 -> (X1) -> X3 -> (W1) -> W4 -> (F4) -> U4 -> U1 -> (B4) -> B3 -> J3 -> J2 -> (H2) -> H1 -> (J4) -> K4 -> (K1) -> K3 -> (P3) -> P1 -> M3 -> D4 -> (D1) -> A1 ->	38.85845591	74332	98013	2.972489119

		A2 -> (G1) -> A2 -> A3 -> (I3) -> I4 -> (R1) -> R4 -> V2 -> [V3]				

Analysis of these results

Key for the following chats: 🟦: BFS 🟥: UCS 🟨: A*





With these charts, we can analyze two things:

- **The effect of the number of target locations over search time, nodes reached and nodes explored:**
  As the difficulty of the problem increases, the number of nodes explored/reached and search time both increase. The search time was significantly larger for test case Hard 4, which took significantly longer than any previous test case. However, the number of nodes explored is not as significant a jump. This tells me the route was very difficult to find with each targeting having a significant number of connections.

- **The effect of the search algorithm used over search time, nodes reached and nodes explored.**
  A* and UCS both outperform BFS across all three measurements, with the degree to which they out perform increasing with the problem difficulty. On simpler test cases, BFS is sufficient to find an optimal path, but on more difficult problems it begins returning sub-optimal solutions. The more interesting comparison is between A* and UCS. The difference in solution costs is small. But A* is significantly more efficient — exploring fewer nodes and finding the optimal path in less time. The heuristic used in A* must allow it to avoid exploring a large number of irrelevant nodes.

New application

What I have learned is that a search algorithm can be used in a variety of applications, not just literal search problems. As long as a problem can be represented as a set of states, with a defined transition function between those states and a goal state, a search algorithm can be applied.

For example, we could use the same type of algorithm to determine the best way to schedule meetings in a building or classes in a high school. The goal would be to minimize hallway traffic and reduce the time it takes for attendees to get to their next meeting/class. We can define this problem using the following criteria:

- Inputs: The class schedule with the list of required attendees, the directory of rooms, and the distance between the rooms.
- State: assigned locations of each classroom during the time slots.
  - History (classroom B), Math (classroom C), Science (classroom A)
- Action: moving from 1 time slot's set of classes to the next time slot's set of classes.
  - 9 am - 11 am: History(classroom B), Math (classroom C), Science (classroom A) →
    11: 30 am - 1 pm: History(classroom B), Art(classroom E), English(classroom A)
- Cost: the number of people who must move between classes * the distance they must travel
- Goal State: Every class is assigned a room for each time slot.

Possible heuristics that could be used: Frequency of classes, size of classes, prioritizing classrooms on the same floor.

Feedback

I enjoyed this assignment. I came away with a good understanding of how the search algorithms work, and analyzing the results helped make that concrete. I particularly enjoyed the part where we had to come up with a new application. This forces you to really understand the algorithms in order to apply them to a new problem space.

Conclusions.

In the course of this assignment, I am walking away with two learnings

- The value of a good heuristic is to significantly improve the efficiency of search
    - The results really demonstrate how A* outperforms the other search algorithms while still finding the optimal path. The huearistic pushes the search in the right direction.
- As the problem becomes more complex, a more sophisticated algorithm becomes valuable.
    - In the simpler test cases, each algorithm performs the same. But when we get to the more difficult cases, A* really shines compared to the other two.